

임건표 담당영역

소스코드 설명

법률 질의응답 RAG 파이프라인과 문서 기반 Hybrid 검색 구현

작성자: 임건표

작성일: 2026-06-29

주요 기술: FastAPI · PostgreSQL · Pinecone · BGE-M3 · Ollama · EXAONE

담당 구현 요약

- 질문을 Dense-Sparse 임베딩으로 변환하고 Pinecone Top-K 검색 결과에 PostgreSQL 원문을 결합
- 일반 질의와 파일 업로드 세션을 구분하고, Hybrid 모드에서 사용자 문서 2건과 공용 법률 3건을 선택
- 이전 대화를 맥락 정보로만 제한하고 현재 참고자료 중심으로 EXAONE 답변을 생성
- 모델 답변의 법령·조문을 Top-K 출처와 대조해 검증 결과와 출처 목록을 함께 반환
- PDF·Office·HWP/HWPX·TXT 문서의 텍스트 추출, OCR, 요약, 청킹 및 벡터 적재 흐름 연동

전체 흐름 WebSocket 질문 수신 → 단독/맥락 검색 → BGE-M3 임베딩 → Pinecone 검색 → RDB 원문 결합 → 참고자료 선택 → 프롬프트 조립 → Ollama/EXAONE 추론 → 환각 검증 및 출처 반환

문서 구성

- 01 RAG 전체 처리 흐름
- 02 멀티턴 검색과 Hybrid 2:3 선택
- 03 Pinecone 검색 결과와 RDB 원문 결합
- 04 BGE-M3 Dense·Sparse 하이브리드 검색
- 05 프롬프트 조립과 참고자료 그라운드링
- 06 환각 검증과 출처 응답
- 07 Ollama 기반 EXAONE 모델 호출
- 08 WebSocket 채팅 연동
- 09 문서 업로드와 OCR/HWP 처리
- 10 단위 테스트 및 구현 요약

핵심 설계값 검색 후보 Top-K 20 · 법률 검색 최소 점수 0.99 · 비법률 최종 차단 점수 1.3 · Dense 70% + Sparse 30% · Hybrid 참고자료 사용자 문서 2건 + 공용 법률 3건

1. RAG 전체 처리 흐름

대상 파일

Back-end/app/services/chat_service.py

핵심 구현

1. 세션에 임베딩 완료 문서가 있으면 general 모드에서 hybrid 모드로 자동 전환
2. 현재 질문만으로 Top-K 20을 먼저 검색하고, 점수가 애매할 때만 직전 질문을 붙여 재검색
3. 최종 Top-1 점수와 법률 도메인 판별을 모두 통과한 경우에만 모델 호출
4. 검색 결과 선택, 프롬프트 조립, EXAONE 응답 생성, 환각 검증을 한 서비스 함수에서 순서대로 지휘

```
105 async def process_chat(  
106     db: Session,  
107     user_id: str,  
108     session_id: str,  
109     question: str,  
110     context_mode: str = "general",  
111     top_k: int = 20,  
112     history_limit: int = 10,  
113     history: list[dict] | None = None,  
114 ) -> ChatAnswerResponse:  
115     """질문 하나를 받아 검색부터 답변 생성까지 실행한다."""  
116  
117     # 파일이 있는 채팅방은 Hybrid 검색으로 전환한다.  
118     if db is not None and context_mode == "general":  
119         has_file = (  
120             db.query(Document)  
121             .filter(  
122                 Document.session_id == session_id,  
123                 Document.status.in_(  
124                     [  
125                         Status.READY,  
126                         Status.EMBEDDED,  
127                     ]  
128                 ),  
129             )  
130             .first()  
131             is not None  
132         )  
133  
134         if has_file:  
135             context_mode = "hybrid"  
136  
137     # 완료된 이전 대화를 조회한다.  
138     if history is None:  
139         history = get_recent_messages(  
140             db=db,  
141             session_id=session_id,  
142             limit=history_limit,  
143         )  
144  
145     # 현재 질문만으로 먼저 검색한다.  
146     standalone_results = search_pinecone(  
147         question=question,  
148         context_mode=context_mode,  
149         user_id=user_id,  
150         top_k=top_k,  
151         db=db,  
152     )
```

```

162     # 점수가 애매하고 이전 대화가 있으면 맥락 검색을 수행한다.
163     if (
164         standalone_top1 <= CONTEXT_SEARCH_MAX
165         and history
166     ):
167         contextual_query = build_contextual_query(
168             question=question,
169             history=history,
170         )
171
172         contextual_results = search_pinecone(
173             question=contextual_query,
174             context_mode=context_mode,
175             user_id=user_id,
176             top_k=top_k,
177             db=db,
178         )
179
180         contextual_top1 = (
181             get_score(contextual_results[0])
182             if contextual_results
183             else 0.0
184         )
185
186         # 맥락 검색이 단독 검색보다 좋을 때만 교체한다.
187         if (
188             contextual_top1 >= LEGAL_MIN_SCORE
189             and contextual_top1 > standalone_top1
190         ):
191             search_results = contextual_results
192
193         final_top1 = (
194             get_score(search_results[0])
195             if search_results
196             else 0.0
197         )
198
199         # 모든 검색이 끝난 뒤 최종 점수로 비법률 질문을 차단한다.
200         if final_top1 <= HARD_REJECT_SCORE:
201             return ChatAnswerResponse(
202                 answer="죄송합니다. 법률 관련 질문에만 답변할 수 있습니다.",
203                 verified=True,
204                 warnings=[],
205                 unverified_refs=[],
206                 sources=[],
207             )
208
209         if not is_legal_domain(search_results):
210             return ChatAnswerResponse(
211                 answer="죄송합니다. 법률 관련 질문에만 답변할 수 있습니다.",
212                 verified=True,
213                 warnings=[],
214                 unverified_refs=[],
215                 sources=[],
216             )
217
218         # Hybrid에서는 사용자 문서 2개와 공용 법률 3개를 선택한다.
219         llm_search_results = select_llm_search_results(
220             search_results=search_results,
221             context_mode=context_mode,
222             document_k=2,
223             public_k=3,
224         )
225
226         messages = assemble_messages(
227             question=question,
228             search_results=llm_search_results,
229             history=history,
230         )
231
232         answer = await generate_answer(messages)
233
234         return build_response(
235             answer,
236             llm_search_results,
237         )

```

구현 의의

- 웹소켓 라우터는 process_chat 하나만 호출하므로 RAG 세부 구현과 전송 계층의 결합도를 낮춤

- 단독 검색을 우선하여 이전 대화가 현재 질문의 검색 결과를 불필요하게 오염시키는 문제를 완화
- 검색 실패와 비법률 질문을 모델 호출 전에 차단해 추론 비용과 무관한 답변 생성을 줄임

2. 멀티턴 검색과 Hybrid 2:3 선택

대상 파일

Back-end/app/services/chat_service.py

핵심 구현

5. 후속 질문의 대상이 생략된 경우 직전 사용자 질문 1개만 현재 질문 앞에 붙여 검색 맥락 보강
6. 이전 모델 답변은 사실 오류가 검색어에 재유입될 수 있어 Pinecone 검색 쿼리에서 제외
7. 파일 업로드 세션에서는 사용자 문서 최대 2건과 공용 법률 자료 최대 3건을 각각 점수순 선택
8. 최종 5개를 다시 전체 점수순으로 정렬해 모델 입력 순서를 일관되게 유지

```

24 def build_contextual_query(
25     question: str,
26     history: list[dict],
27 ) -> str:
28     """후속 질문이면 직전 질문 하나를 검색어에 붙인다."""
29
30     if not history:
31         return question
32
33     previous_question = (
34         history[0].get("question") or ""
35     ).strip()
36
37     if previous_question and previous_question != question:
38         return f"{previous_question} {question}"
39
40     return question

```

```

52 def select_llm_search_results(
53     search_results: list[dict],
54     context_mode: str,
55     document_k: int = 2,
56     public_k: int = 3,
57 ) -> list[dict]:
58     """
59     파일 업로드 세션에서는:
60     - 사용자 문서 최대 2개
61     - 공용 법률 최대 3개
62
63     일반 세션에서는 점수순 상위 5개를 사용한다.
64     """
65
66     total_k = document_k + public_k
67
68     if context_mode != "hybrid":
69         return search_results[:total_k]
70
71     document_results = [
72         result
73         for result in search_results
74         if result.get("source_type") == "document"
75     ]
76
77     public_results = [
78         result
79         for result in search_results
80         if result.get("source_type") == "legal_vector"

```

```

81     ]
82
83     document_results.sort(
84         key=get_score,
85         reverse=True,
86     )
87     public_results.sort(
88         key=get_score,
89         reverse=True,
90     )
91
92     selected = (
93         document_results[:document_k]
94         + public_results[:public_k]
95     )
96
97     selected.sort(
98         key=get_score,
99         reverse=True,
100    )
101
102    return selected

```

구현 의의

- 개인 문서에만 편향되거나 공용 법률 자료에만 편향되지 않는 고정 비율의 Hybrid 참고자료 구성
- 멀티턴 맥락은 살리면서 이전 답변의 잘못된 조문·수치가 검색 결과에 반복 반영되는 위험을 축소

3. Pinecone 검색 결과와 RDB 원문 결합

대상 파일

Back-end/app/services/vector_service.py

핵심 구현

9. Pinecone 결과를 id, score, source_type, metadata의 공통 구조로 정규화
10. metadata에 source_text가 있으면 먼저 text로 승격하고, 없으면 vector_id로 PostgreSQL 원문 조회
11. 공용 법률은 chunk_dataset, 사용자 문서는 Chunk 모델에서 한 번의 IN 조회로 본문과 article을 수집
12. general은 public namespace, document는 user namespace, hybrid는 두 namespace를 모두 검색
13. 결합된 chunk_text와 article을 metadata에 넣은 뒤 프롬프트 서비스로 전달

```

113 def _attach_chunk_text(
114     db: Session | None,
115     results: list[dict],
116 ) -> list[dict]:
117     """
118     검색 결과의 vector_id를 이용해 RDB 본문과 조문을 결합한다.
119     """
120
121     # Pinecone metadata에 source_text가 있으면 우선 사용한다.
122     for result in results:
123         metadata = result.get("metadata") or {}
124
125         if (
126             not metadata.get("text")
127             and metadata.get("source_text")
128         ):
129             metadata["text"] = metadata["source_text"]
130
131         result["metadata"] = metadata
132

```

```

133     if db is None or not results:
134         return results
135
136     public_ids: set[uuid.UUID] = set()
137     document_ids: set[uuid.UUID] = set()
138
139     # 공용 법률과 사용자 문서의 vector_id를 분리한다.
140     for result in results:
141         metadata = result.get("metadata") or {}
142
143         raw_id = (
144             metadata.get("vector_id")
145             or result.get("id")
146         )
147
148         vector_id = _to_uuid(raw_id)
149
150         if vector_id is None:
151             continue
152
153         if result.get("source_type") == "legal_vector":
154             public_ids.add(vector_id)
155         else:
156             document_ids.add(vector_id)
157
158     public_texts: dict[str, str] = {}
159     public_articles: dict[str, str] = {}
160
161     # 공용 법률 본문과 조문을 조회한다.
162     if public_ids:
163         statement = text(
164             """
165             SELECT vector_id, chunk_text, article
166             FROM chunk_dataset
167             WHERE vector_id::text IN :vector_ids
168             """
169         ).bindparams(
170             bindparam(
171                 "vector_ids",
172                 expanding=True,
173             )
174         )
175
176     rows = db.execute(
177         statement,
178         {
179             "vector_ids": [
180                 str(value)
181                 for value in public_ids
182             ]
183         },
184     ).mappings().all()

```

```

231     # 조회한 본문과 조문을 검색 결과 metadata에 결합한다.
232     for result in results:
233         metadata = result.get("metadata") or {}
234
235         raw_id = (
236             metadata.get("vector_id")
237             or result.get("id")
238         )
239
240         vector_id = _to_uuid(raw_id)
241
242         if vector_id is None:
243             continue
244
245         key = str(vector_id)
246
247         if result.get("source_type") == "legal_vector":
248             chunk_text = public_texts.get(key)
249             article = public_articles.get(key)
250         else:
251             chunk_text = document_texts.get(key)
252             article = document_articles.get(key)
253
254         if chunk_text and not metadata.get("text"):
255             metadata["text"] = chunk_text
256
257         if article and not metadata.get("article"):

```

```
258         metadata["article"] = article
259
260         result["metadata"] = metadata
261
262     return results
```

```
313     # 공용 법률 자료 검색
314     if context_mode in (
315         "general",
316         "hybrid",
317     ):
318         public_matches = pinecone_client.query(
319             dense_vector=dense_vector,
320             sparse_vector=sparse_vector,
321             namespace=pinecone_client.public_namespace(),
322             top_k=top_k,
323             filter=None,
324         )
325
326         for match in public_matches:
327             results.append(
328                 _normalize_result(
329                     match,
330                     namespace="public",
331                 )
332             )
333
334     # 사용자 문서 검색
335     if (
336         context_mode in (
337             "document",
338             "hybrid",
339         )
340         and user_id
341     ):
342         user_namespace = (
343             pinecone_client.user_namespace(
344                 str(user_id)
345             )
346         )
347
348         user_matches = pinecone_client.query(
349             dense_vector=dense_vector,
350             sparse_vector=sparse_vector,
351             namespace=user_namespace,
352             top_k=top_k,
353             filter=None,
354         )
355
356         for match in user_matches:
357             results.append(
358                 _normalize_result(
359                     match,
360                     namespace=user_namespace,
361                 )
362             )
363
364     if context_mode not in (
365         "general",
366         "document",
367         "hybrid",
368     ):
369         raise ValueError(
370             f"지원하지 않는 context_mode입니다: {context_mode}"
371         )
372
373     results.sort(
374         key=_get_score,
375         reverse=True,
376     )
377
378     # 일반 또는 사용자 문서 검색은 Top-K만 남긴다.
379     # Hybrid는 사용자·공용 후보를 모두 유지한다.
380     if context_mode != "hybrid":
381         results = results[:top_k]
382
383     return _attach_chunk_text(
384         db=db,
385         results=results,
386     )
```


구현 의의

- 벡터 DB에는 검색에 필요한 벡터와 메타데이터를 두고, 긴 원문은 RDB에서 조회하는 분리 구조 구현
- Top-K에 포함된 vector_id만 IN 조건으로 조회하므로 전체 청크 테이블을 애플리케이션 메모리로 읽지 않음
- 공용/개인 namespace와 RDB 테이블의 대응 관계를 source_type으로 명확히 유지

4. BGE-M3 Dense·Sparse 하이브리드 검색

대상 파일

```
Back-end/app/db/vector/embedding.py
Back-end/app/db/vector/client.py
```

핵심 구현

- 14.BAAI/bge-m3 모델을 최초 1회 로드하고 전역 캐시로 재사용
- 15.질문에서 Dense 1024차원 벡터와 Sparse lexical weight를 동시에 생성
- 16.Sparse 임계값 이하 토큰을 제거하고 최대값 기준으로 정규화하여 노이즈를 완화
- 17.Pinecone 질의 시 Dense 70%, Sparse 30%로 가중치를 적용
- 18.public과 user_{user_id} namespace를 분리해 공용·개인 데이터 격리

```
29 def get_model() -> BGEM3FlagModel:
30     global _model
31
32     # print("모델 가져오기")
33     if _model is None:
34         _model = BGEM3FlagModel(
35             settings.EMBEDDING_MODEL,
36             use_fp16=True,
37             device=EMBEDDING_DEVICE,
38         )
39     # print("모델 가져오기 성공")
40     return _model
41
42
43 # — 임베딩 결과 데이터클래스 —————
44 @dataclass
45 class EmbeddingResult:
46     dense: list[float] # 1024차원
47     sparse: dict # {"indices": [...], "values": [...]}
48     text: str
```

```
58 # — sparse 파싱 + 후처리 —————
59 def _parse_sparse(raw_sparse: dict) -> dict:
60     """
61     BGE-M3 lexical_weights -> Pinecone sparse 포맷으로 변환
62     1. SPARSE_THRESHOLD 이하 토큰 제거 (노이즈 필터)
63     2. values 정규화 (max=1.0 기준) -> dense와 스케일 맞춤
64     """
65     indices: list[int] = []
66     values: list[float] = []
67
68     if isinstance(raw_sparse, dict):
69         if "indices" in raw_sparse and "values" in raw_sparse:
70             # {"indices": [...], "values": [...]} 형태
71             for idx, val in zip(raw_sparse["indices"], raw_sparse["values"]):
```

```

72         if float(val) > SPARSE_THRESHOLD: # ✅ 노이즈 필터
73             indices.append(int(idx))
74             values.append(float(val))
75     else:
76         # {"3412": 0.45, ...} 일반 형태
77         for k, v in raw_sparse.items():
78             if float(v) > SPARSE_THRESHOLD: # ✅ 노이즈 필터
79                 indices.append(int(k))
80                 values.append(float(v))
81     else:
82         # dict-like 객체 안전 처리
83         try:
84             if hasattr(raw_sparse, "indices") and hasattr(raw_sparse, "values"):
85                 for idx, val in zip(
86                     getattr(raw_sparse, "indices"),
87                     getattr(raw_sparse, "values"),
88                 ):
89                     if float(val) > SPARSE_THRESHOLD:
90                         indices.append(int(idx))
91                         values.append(float(val))
92         except Exception:
93             pass
94
95     # ✅ sparse values 정규화 (max=1.0 기준)
96     # dense는 BGE-M3가 L2 정규화하므로 sparse도 스케일 맞춰야 기여도 안정적
97     if values:
98         max_val = max(values)
99         if max_val > 0:
100             values = [v / max_val for v in values]
101
102     return {"indices": indices, "values": values}

```

```

45 # 네임스페이스로 구분 헬퍼
46 def public_namespace() -> str:
47     return "public"
48
49 def user_namespace(user_id: str) -> str:
50     return f"user_{user_id}"
51
52 # Upsert
53 def upsert(vectors: list[dict], namespace:str)-> dict:
54     return get_index().upsert(vectors=vectors, namespace=namespace)
55
56 # Query
57 def query(dense_vector: list[float], sparse_vector:dict, namespace:str, top_k:int=5, filter:
58     dict | None=None, alpha:float=0.7) -> list[dict]:
59     # dense_vector값과 곱해주는 함수입니다. v가 dense_vector값 이는 70퍼 할당한다는 의미입니다.
60     # sparse_vector는 자동으로 30퍼만할당.
61     scaled_dense = [v * alpha for v in dense_vector]
62     scaled_sparse = {
63         # Sparse 벡터에서 어떤 토큰(단어)인지 가리키는 번호. indices랑 values하나가 쌍.
64         "indices": sparse_vector["indices"],
65         "values": [v * (1-alpha) for v in sparse_vector["values"]],
66     }
67
68     response = get_index().query(
69         vector=scaled_dense,
70         sparse_vector=scaled_sparse,
71         namespace=namespace,
72         top_k=top_k,
73         filter=filter,
74         include_metadata=True,
75     )
76
77     # 2. [수정] 최신 Pinecone SDK의 QueryResponse 객체는 딕셔너리처럼 .get()을 쓰면 에러가 날
78     # 수 있습니다.
79     # 안전하게 점 접근 방식(response.matches)을 쓰거나 객체 구조에 맞춰 처리해야 합니다.
80     if hasattr(response, "matches"):
81         return [match.to_dict() for match in response.matches]
82     return response.get("matches", [])

```

구현 의의

- Dense 의미 유사도와 Sparse 법률 용어 일치도를 결합하여 조문명·법률용어가 중요한 검색 특성을 반영

- 임베딩 모델 캐싱으로 질문마다 모델을 다시 로드하는 지연을 방지
- 사용자별 namespace로 업로드 문서가 다른 사용자의 검색 결과에 섞이지 않도록 구성

5. 프롬프트 조립과 참고자료 그라운드

대상 파일

```
Back-end/app/services/prompt_service.py
Back-end/app/services/prompts.py
```

핵심 구현

19. 이전 대화는 현재 질문의 대상을 이해하기 위한 참고용으로만 표시하고 답변은 최대 120자로 제한
20. 참고자료 본문은 text → source_text → qa_text 순서로 선택
21. article과 law_name/category를 결합해 모델이 조문 출처를 구분할 수 있는 형식으로 변환
22. system prompt와 하나의 user 메시지 안에 이전 맥락, 참고자료, 현재 질문을 명확한 구획으로 조립
23. 법령명·조문·판례번호·형량·벌금·비율 등은 참고자료에 있을 때만 사용하도록 제한

```
4 def format_history_context(history: list[dict]) -> str:
5     """
6     이전 대화는 현재 질문의 맥락 파악용으로만 전달한다.
7     법적 근거로 쓰지 못하게 프롬프트에서 명확히 제한한다.
8     """
9
10    if not history:
11        return ""
12
13    lines = [
14        "[이전 대화 맥락]",
15        "아래 이전 대화는 사용자의 이어지는 질문을 이해하기 위한 참고용입니다.",
16        "이전 대화의 답변 내용은 법적 근거로 사용하지 마세요.",
17        "법령명, 조문번호, 수치, 처벌 내용, 판례번호는 반드시 [참고자료]에 있는 내용만 근거로",
18        "사용하세요.",
19    ]
20
21    # history는 최신순이라고 가정하므로 오래된 것부터 보여준다.
22    for i, turn in enumerate(reversed(history), start=1):
23        question = (turn.get("question") or "").strip()
24        answer = (turn.get("answer") or "").strip()
25
26        if question:
27            lines.append(f"{i}. 이전 질문: {question}")
28
29        # 이전 답변은 맥락용으로만 짧게 전달한다.
30        if answer:
31            answer_summary = answer[:120].replace("\n", " ").strip()
32            lines.append(f"이전 답변 참고: {answer_summary}")
33
34    return "\n".join(lines)
```

```

38 def format_search_results(search_results: list[dict]) -> str:
39     if not search_results:
40         return "[참고자료]\n관련 자료를 찾을 수 없습니다."
41
42     lines = [
43         "[참고자료]",
44         "아래 참고자료만 근거로 답변하세요.",
45         "법령명이나 조문번호를 새로 생성하지 마세요.",
46         "답변 마지막 줄에 사용한 조문을 '근거: 법령명 제○조' 형식으로 표시하세요.",
47         "근거에는 반드시 참고자료의 [조문] 필드에 있는 값만 사용하세요.",
48         "",
49     ]
50
51     for i, result in enumerate(search_results, start=1):
52         metadata = result.get("metadata") or {}
53
54         content = (
55             metadata.get("text")
56             or metadata.get("source_text")
57             or metadata.get("qa_text")
58             or ""
59         )
60
61         article = (metadata.get("article") or "").strip()
62         law_name = (
63             metadata.get("law_name")
64             or metadata.get("category")
65             or ""
66         ).strip()
67
68         content = str(content).strip() or "본문 내용 없음"
69
70         display_article = article
71         if article and law_name and law_name not in article:
72             display_article = f"{law_name} {article}"
73
74         if display_article:
75             lines.append(f"{i}. [조문: {display_article}] [본문]\n{content}")
76         elif law_name:
77             lines.append(f"{i}. [법령명: {law_name}] [조문: 확인 불가] [본문]\n{content}")
78         else:
79             lines.append(f"{i}. [조문: 확인 불가] [본문]\n{content}")
80
81     return "\n".join(lines)

```

```

85 def assemble_messages(
86     question: str,
87     search_results: list[dict],
88     history: list[dict],
89 ) -> list[dict]:
90
91     history_context = format_history_context(history)
92     search_str = format_search_results(search_results)
93
94     user_parts = []
95
96     if history_context:
97         user_parts.append(history_context)
98
99     user_parts.append(search_str)
100     user_parts.append(f"[현재 질문]\n{question}")
101
102     user_content = "\n\n".join(user_parts)
103
104     messages = [
105         {"role": "system", "content": SYSTEM_PROMPT},
106         {"role": "user", "content": user_content},
107     ]
108
109     return messages

```

```

4  [핵심 원칙]
5  - 반드시 현재 질문에 직접 답변하세요.
6  - 답변 내용은 오직 [참고자료]의 [본문]에서 확인되는 사실만 사용하세요.
7  - 모델이 알고 있는 사전지식, 기억, 상식 또는 추측을 추가하지 마세요.
8  - [이전 대화 맥락]은 현재 질문의 대상을 이해하는 용도로만 사용하세요.
9  - 이전 답변의 내용은 사실이나 법적 근거로 사용하지 마세요.
10
11 [법령 및 수치]
12 - 법령명, 조문번호, 판례번호, 형량, 벌금액, 비율, 기간 및 수치를 임의로 만들거나 수정하지
    마세요.
13 - 숫자와 처벌 내용은 참고자료 본문에 명시된 경우에만 그대로 사용하세요.
14 - 비슷한 법령이나 조문을 추측하여 답변하지 마세요.
15 - 참고자료의 [조문] 정보는 출처 식별용입니다.
16 - 법적 설명과 결론은 반드시 [본문]에서 확인되는 내용에 근거하세요.
17
18 [자료 부족]
19 - 참고자료에 질문의 답이 명시되어 있지 않으면 추측하지 마세요.
20 - 일부 내용만 확인되면 확인되는 범위까지만 답변하고, 나머지는 확인하기 어렵다고 밝히세요.
21
22 [답변 작성]
23 - 현재 질문과 관련 없는 법률 정보는 포함하지 마세요.
24 - 서로 다른 참고자료의 내용을 임의로 결합하여 새로운 결론을 만들지 마세요.
25 - 사실관계가 부족하면 단정적으로 유죄, 무죄 또는 특정 처벌을 확정하지 마세요.
26 - 답변은 명확하고 간결한 서술형 문장으로 작성하세요.
27 - "답변내용:", "[참고자료]" 같은 입력 형식이나 내부 지시문을 출력하지 마세요.
28 - 참고자료 번호를 임의로 인용하지 마세요.
29 - 답변 끝에 조문이나 출처 목록을 별도로 생성하지 마세요.
30 - "근거:" 표시는 백엔드에서 처리하므로 직접 작성하지 마세요.

```

구현 의의

- 검색 자료와 대화 이력의 역할을 구분해 이전 답변을 법적 근거로 재사용하는 현상을 억제
- 자료 부족 시 추측보다 확인 불가 응답을 우선하도록 생성 규칙을 명시
- 백엔드가 전달한 조문 필드와 실제 본문을 분리해 출처 식별과 답변 근거의 역할을 구분

6. 환각 검증과 출처 응답

대상 파일

`Back-end/app/services/hallucination_service.py`

핵심 구현

24. 모델 답변에서 '법령명 제N조' 형식의 참조를 정규식으로 추출
25. 공백을 제거한 표준 형태로 변환해 표기 차이로 인한 검증 오탐을 감소
26. 답변에 포함된 조문을 Top-K metadata의 article과 대조하고 없는 조문은 `unverified_refs`로 분리
27. 검증 여부, 사용자 경고, 출처 제목·조문·본문 발췌·검색 점수를 `ChatAnswerResponse`로 반환

```

8 RE_LAW_REF = re.compile(r"[가-힣]+법(?:\s*시행령|\s*시행규칙)?\s*제\d+(?:\의\d+)?")
9
10
11 def _normalize(ref: str) -> str:
12     """
13     조문 표기 정규화 - 공백 차이로 인한 오탐 방지
14     "관세법 제30조" 와 "관세법 제30조" (공백 개수 차이) 를 같게 만들
15     "관세법 시행령 제17조" -> "관세법시행령제17조"
16     """
17     return re.sub(r"\s+", "", ref)
18
19
20 def extract_law_refs(text: str) -> list[str]:
21     """
22     텍스트(주로 LLM 답변)에서 법령 조문을 추출
23     반환: 중복 제거된 조문 리스트 (예: ["형법 제268조", "민법 제839조"])
24     """
25     refs = RE_LAW_REF.findall(text)
26     # 순서 유지하면서 중복 제거
27     seen = set()
28     unique = []
29     for r in refs:
30         key = _normalize(r)
31         if key not in seen:
32             seen.add(key)
33             unique.append(r)
34     return unique

```

```

64 def verify_answer(answer: str, search_results: list[dict]) -> dict:
65     """
66     LLM 답변의 조문이 Top-K에 실제로 있는지 검증 (A방식: 표시만, 답변 미수정)
67
68     반환:
69     {
70         "verified": bool,          # 할각 없으면 True
71         "answer_refs": [...],      # 답변에서 추출된 조문 전체
72         "verified_refs": [...],    # Top-K에서 확인된 조문
73         "unverified_refs": [...],  # Top-K에 없는 조문 (할각 의심)
74         "warnings": [...],        # 사용자용 경고 문구
75     }
76     """
77     # 답변에서 조문 추출
78     answer_refs = extract_law_refs(answer)
79
80     # 답변에 조문이 아예 없으면 -> 검증할 것 없음 -> 통과
81     if not answer_refs:
82         return {
83             "verified": True,
84             "answer_refs": [],
85             "verified_refs": [],
86             "unverified_refs": [],
87             "warnings": [],
88         }
89
90     # Top-K의 정답 조문 집합
91     valid_refs = _collect_topk_refs(search_results)
92
93     verified = []
94     unverified = []
95     for ref in answer_refs:
96         if _normalize(ref) in valid_refs:
97             verified.append(ref)
98         else:
99             unverified.append(ref)
100
101     # 경고 문구 생성 (조항 꼭 집기)
102     warnings = [
103         f"'{ref}'는 참고 자료에서 직접 확인되지 않았습니다. 정확하지 않을 수 있습니다."
104         for ref in unverified
105     ]
106
107     return {
108         "verified": len(unverified) == 0,
109         "answer_refs": answer_refs,
110         "verified_refs": verified,
111         "unverified_refs": unverified,
112         "warnings": warnings,
113     }

```

```

116 def format_sources(search_results: list[dict]) -> list[dict]:
117     """
118     Top-K 검색 결과 → 사용자에게 보여줄 출처 리스트
119
120     실제 판례 데이터 구조 기준 (category, text, law_date, article)
121     제목 필드가 없으므로 category + 본문 발췌로 표시
122     """
123     sources = []
124     for r in search_results:
125         metadata = r.get("metadata", {})
126         source_type = r.get("source_type", "")
127
128         if source_type == "document":
129             # 개인 업로드 문서
130             title = metadata.get("filename", "업로드 문서")
131         else:
132             # 공용 법령/판례 - 제목 없으면 category로
133             title = metadata.get("doc_title") or metadata.get("category", "법령/판례")
134
135         text = metadata.get("text") or metadata.get("qa_text", "")
136         excerpt = text[:120].replace("\n", " ").strip()
137         if len(text) > 120:
138             excerpt += "..."
139
140         sources.append({
141             "title": title,
142             "category": metadata.get("category"),
143             "article": metadata.get("article"),
144             "law_date": metadata.get("law_date"),
145             "excerpt": excerpt,
146             "text": text,
147             "score": round(r.get("score", 0), 4),
148         })
149
150     return sources
151
152
153 def build_response(answer: str, search_results: list[dict]) -> ChatAnswerResponse:
154     verification = verify_answer(answer, search_results)
155     sources = format_sources(search_results)
156
157     return ChatAnswerResponse(
158         answer=answer,
159         verified=verification["verified"],
160         warnings=verification["warnings"],
161         unverified_refs=verification["unverified_refs"],
162         sources=sources,
163     )

```

구현 의의

- 생성 답변을 그대로 신뢰하지 않고 검색 결과와 다시 대조하는 후처리 검증 계층 추가
- 검증 실패 시 답변을 숨기기보다 어떤 조문이 자료에서 확인되지 않았는지 명시적으로 전달
- 프론트엔드가 출처 목록과 검색 점수를 표시할 수 있는 구조화 응답 제공

7. Ollama 기반 EXAONE 모델 호출

대상 파일

```

Back-end/app/core/config.py
Back-end/app/services/service.py

```

핵심 구현

28. 답변 모델, 요약 모델, 임베딩 모델을 설정에서 역할별로 분리

- 29. 법률 답변 모델 law-exaone-7.8b-q6를 Ollama /api/chat 엔드포인트로 비동기 호출
- 30. temperature 0, top_p 0.85, repeat_penalty 1.15로 답변의 무작위성과 반복을 제어
- 31. num_ctx 16,384와 num_predict 300으로 멀티턴·참고자료 입력과 답변 길이를 관리
- 32. HTTP 오류는 raise_for_status로 상위 계층에 전달하고 최대 600초 timeout 적용

```

12 class Settings(BaseSettings):
13     PROJECT_NAME: str = "법률 챗봇"
14
15     # 모델설정
16     OLLAMA_BASE_URL: str = Field(default="http://localhost:11434") #모델 url
17     # 역할에 따라 모델 분리
18     # str은 힌팅식 python 문법입니다. typescript처럼 타입을 미리 정의를 해주는방식
19     SUMMARIZE_MODEL: str = Field(default="exaone3.5:2.4b")
20     EMBEDDING_MODEL: str = Field(default="BAAI/bge-m3") # 임베딩용
21     RAG_MODEL: str = "law-exaone-7.8b-q6" # RAG 응답용
22     #임베딩모델 설정
23     SPARSE_THRESHOLD: float = Field(default=...)
24     EMBEDDING_DEVICE: str = Field(default=...)
25     EMBEDDING_BATCH: int = Field(default=...)

```

```

11 async def generate_answer(messages : list[dict]):
12     # 함수의 역할 설명
13     """
14     Ollama에 메시지 리스트를 보내고 답변 문자열을 반환
15     messages 형식 : [{"role": "system", "content": "..."}, {"role": "user", "content": "..."}]
16     """
17     # Ollama에게 보낼 요청
18     payload = {
19         # config에 세팅해놓은 모델 선택 (학습시킨 모델 가져옴)
20         "model": settings.RAG_MODEL,
21         # 실제로 모델에게 보낼 대화 내용
22         "messages": messages,
23         # 답변을 한번에 받을지, 조금씩 받을지 정하는 곳 False > 한 번에
24         "stream": False,
25         # 모델 답변 스타일 설정
26         "options": {
27             # 모델의 창의성, 랜덤성 조정 0.3으로 보수적으로 답변 (법률 도메인이라 선택)
28             "temperature": 0,
29             # 모델이 다음 단어를 고를 때 후보 범위 설정
30             # 너무 낮으면 단조롭고, 너무 높으면 답변이 흔들릴 수 있다
31             "top_p": 0.85,
32             # 모델이 반복되는 말을 억제하는 설정
33             "repeat_penalty": 1.15,
34             "num_thread": 4,
35             "num_ctx": 16384, # 그릇 크기 → 10턴 안 찰림
36             "num_predict": 300,
37         }
38     }
39     # Ollama와 통신하기 위한
40     # async with : 작업이 끝나면 자동으로 정리
41     # httpx.AsyncClient : 비동기 방식으로 HTTP 요청을 보냄
42     # timeout=300.0 : 요청을 최대 300초까지 기다리겠다
43     async with httpx.AsyncClient(timeout=600.0) as client:
44         # await : 비동기 함수 결과를 기다림
45         # client.post : POST 요청을 보내는 코드
46         response = await client.post(
47             # Ollama의 채팅 API로 요청을 보냄]
48             # /api/chat → 채팅 형식으로 답변 생성
49             f"{settings.OLLAMA_BASE_URL}/api/chat",
50             # payload를 json형태로 보내겠다
51             json = payload
52         )
53         # HTTP 요청이 실패했는지 확인
54         response.raise_for_status()
55         # Ollama가 보내준 응답을 파이썬 딕셔너리로 바꿈
56         data = response.json()
57         # 답변 내용만 반환
58         return data["message"]["content"]

```


구현 의의

- 로컬 Ollama와 Docker 내부 Ollama를 OLLAMA_BASE_URL 환경변수만으로 전환 가능
- 모델 이름을 config에 집중시켜 코드 수정 없이 배포 환경의 모델을 교체할 수 있도록 구성
- 추론 옵션을 보수적으로 설정해 법률 답변의 재현성과 일관성을 우선

8. WebSocket 채팅 연동

대상 파일

Back-end/app/routes/websocket/chat.py

핵심 구현

33. 사용자 질문을 DB 메시지로 먼저 저장한 뒤 process_chat으로 RAG 파이프라인 호출
34. 생성된 답변을 같은 메시지 레코드에 반영하고 채팅방 갱신 시각 업데이트
35. 최종 WebSocket 응답에 질문, 답변, sources, 질문·답변 시각을 구조화해 전송
36. AI 답변 생성 성공 로그에 사용자·세션·trace_id를 남겨 추적 가능성 확보

```

183         # 메시지 저장
184         msg = Message(
185             session_id=chat_session.session_id,
186             user_id=user.user_id,
187             question=message_text,
188             question_at=datetime.now(timezone.utc),
189             answer = "",
190             answer_at=datetime.now(timezone.utc)
191         )
192
193         db.add(msg)
194         db.commit()
195         db.refresh(msg)
196
197         # 웹소켓 파이프라인 연결
198         response = await process_chat(
199             db=db,
200             user_id=str(user.user_id),
201             session_id=str(chat_session.session_id),
202             question=message_text,
203         )
204         ai_answer = response.answer
205
206         msg.answer = ai_answer
207         msg.answer_at = datetime.now(timezone.utc)
208
209         # 채팅방 updated_at 갱신
210         if hasattr(chat_session, "updated_at"):
211             chat_session.updated_at = datetime.now(timezone.utc)
212
213         db.add(msg)
214         db.commit()
215         db.refresh(msg)
216
217         create_log(
218             db,
219             level=Level.INFO,
220             endpoint="/ws/chat",
221             method="WS",
222             status_code=200,
223             message=f"AI 답변 생성 성공 - message_id={msg.message_id}",
224             user_id=user.user_id,
225             session_id=chat_session.session_id,
226             trace_id=trace_id,
227         )

```

```

228
229         await websocket.send_json({
230             "type": "message",
231             "session_id": str(chat_session.session_id),
232             "message_id": str(getattr(msg, "message_id", "")),
233             "question": msg.question,
234             "answer": msg.answer,
235             "sources": response.sources,
236             "question_at": msg.question_at.isoformat()
237             if msg.question_at
238             else None,
239             "answer_at": msg.answer_at.isoformat()
240             if msg.answer_at
241             else None,
242         })

```

구현 의의

- 실시간 채팅 전송 계층과 RAG 서비스 계층을 process_chat 인터페이스로 분리
- 답변과 함께 출처를 보내 프론트엔드에서 근거 확인 UI를 구성할 수 있도록 지원
- 질문·답변 저장과 로그 기록을 포함해 대화 이력 및 장애 추적 기반 마련

9. 문서 업로드와 OCR/HWP 처리

대상 파일

```

Back-end/app/routes/chat/document.py
Back-end/app/db/vector/document_pipeline.py
Back-end/app/db/vector/read_ocr.py

```

핵심 구현

- 37.PDF, PPT/PPTX, DOCX, XLSX는 PyMuPDF 기반 native text와 이미지 영역 OCR을 결합
- 38.HWP/HWPX는 임시 파일로 저장해 rhwp로 파싱하고, 구형 HWP 실패 시 사용자 안내 반환
- 39.TXT는 UTF-8, CP949, EUC-KR 순서로 디코딩해 한글 파일 호환성 확보
- 40.업로드 문서를 요약·분류하고 법률 문서 여부를 검증한 뒤 청킹 및 사용자 namespace에 벡터 적재
- 41.OCR은 페이지 전체가 아닌 이미지 block만 처리하고 native text 중복을 제거해 속도와 품질 개선

]

```

33 def detect_doc_type(filename: str) -> DocType:
34     ext = filename.split(".")[1].lower() if "." in filename else ""
35     mapping = {
36         "pdf": DocType.PDF, "pptx": DocType.PPTX, "docx": DocType.DOCX,
37         "hwp": DocType.HWP, "hwpx": DocType.HWPX, "ppt": DocType.PPT,
38         "txt": DocType.TXT, "xlsx": DocType.XLSX,
39     }
40     return mapping.get(ext, DocType.UNKNOWN)
41
42
43 def extract_text_from_file(file_bytes: bytes, filename: str) -> str:
44     doc_type = detect_doc_type(filename)
45
46     if doc_type == DocType.UNKNOWN:
47         raise ValueError(f"지원하지 않는 파일 형식입니다. (파일명: {filename})")
48
49     # — [분기 1] PDF, PPTX, DOCX, XLSX, PPT → PyMuPDF + OCR —
50     if doc_type in [DocType.PDF, DocType.PPTX, DocType.DOCX, DocType.XLSX, DocType.PPT]:
51         try:
52             # ✅ 수정 pipeline에서 bytes를 그대로 넘기기
53             return process_pdf(file_bytes, filetype=doc_type.value)

```

```

54         except Exception as e:
55             raise RuntimeError(f"[{doc_type.value.upper()}] OCR 텍스트 추출 오류: {e}")
56
57     # — [분기 2] HWP / HWPX → rhwp 직접 파싱 —
58     elif doc_type in [DocType.HWP, DocType.HWPX]:
59         try:
60             import rhwp
61             import tempfile, os
62
63             suffix = f".{doc_type.value}"
64             with tempfile.NamedTemporaryFile(suffix=suffix, delete=False) as tmp:
65                 tmp.write(file_bytes)
66                 tmp_path = tmp.name
67
68             try:
69                 doc = rhwp.parse(tmp_path)
70                 text = doc.extract_text()
71                 if not text or not text.strip():
72                     raise RuntimeError(
73                         "HWP 3.0 이하 포맷은 지원하지 않습니다. "
74                         "HWPX 형식으로 변환 후 다시 업로드해주세요."
75                     )
76                 return text
77             finally:
78                 os.unlink(tmp_path)
79
80         except RuntimeError:
81             raise
82         except Exception as e:
83             raise RuntimeError(f"[{doc_type.value.upper()}] 텍스트 추출 오류: {e}")
84
85     # — [분기 3] TXT —
86     elif doc_type == DocType.TXT:
87         for encoding in ["utf-8", "cp949", "euc-kr"]:
88             try:
89                 return file_bytes.decode(encoding)
90             except UnicodeDecodeError:
91                 continue
92         raise RuntimeError("[TXT] 지원하는 인코딩 형식이 아닙니다.")
93
94     raise RuntimeError(f"분기 처리 누락된 문서 타입: {doc_type}")

```

```

184     # 텍스트 추출 → 요약
185     with open(file_path, "rb") as f:
186         file_bytes = f.read()
187
188     # 텍스트 추출
189     extracted_text = ""
190     try:
191         extracted_text = extract_text_from_file(file_bytes, original_filename)
192     except Exception as e:
193         print("텍스트 추출 실패:", e)
194
195     if os.path.exists(file_path):
196         os.remove(file_path)
197
198     message = str(e)
199     if "UNSUPPORTED_HWPML" in message or "HWPML 2.1" in message or "HWP 3.0" in
200         message:
201         raise HTTPException(
202             status_code=status.HTTP_400_BAD_REQUEST,
203             detail="HWP 3.0 이하 포맷은 지원하지 않습니다.\nHWPX 형식으로 변환 후 다시
204             업로드해주세요.",
205         )
206
207     raise HTTPException(
208         status_code=status.HTTP_400_BAD_REQUEST,
209         detail="문서 텍스트 추출에 실패했습니다.",
210     )
211
212     meta = None
213     summary = ""
214     if extracted_text:
215         try:
216             meta = summarize_document(extracted_text)
217             summary = meta.summary
218             print("=== 요약 결과 ===")
219             print("category:", meta.category)
220             print("law_date:", meta.law_date)
221             print("law_name:", meta.law_name)

```

```

220         print("summary:", meta.summary[:100]) # 너무 길면 100자만
221     except Exception as e:
222         print("요약 실패:", e)
223         summary = ""
224
225     # 법률 문서 아니면 파일 삭제 후 에러 반환
226     if meta is None or meta.category not in (LAW_CATEGORIES | LAW_UNSTRUCTURED):
227         os.remove(file_path) # 저장된 파일도 정리
228         raise HTTPException(
229             status_code=status.HTTP_400_BAD_REQUEST,
230             detail="법률 관련 문서만 업로드 가능합니다.",
231         )
232
233     # 추후 요약LLM도 생성 후 summary에 적용 시키기
234     # Websocket 연동도 같이 진행
235     document = Document(
236         user_id=current_user.user_id,
237         session_id=session_id,
238         file_name=original_filename,
239         file_ext=file_type,
240         file_size_bytes=file_size,
241         storage_url=file_path,
242         status=Status.CHUNKED,
243         category=Category(meta.category),
244         summary=summary,
245     )
246
247     db.add(document)
248     db.commit()
249     db.refresh(document)
250
251     # 청킹테이블에 적재.
252     await index_document(
253         text=extracted_text,
254         document_id=document.document_id,
255         user_id=current_user.user_id,
256         db=db,
257         category=meta.category,
258         law_name=meta.law_name or "",
259         law_date=meta.law_date or "",
260         pre_chunked=meta.chunks, # ← 청킹 재사용 (두 번 청킹 방지)
261         is_public=False,
262     )
263     document.status = Status.EMBEDDED
264     db.commit()

```

```

507 def process_pdf(pdf_bytes: bytes, zoom=2, filetype: str = "pdf"):
508     """
509     PDF 전체 처리 함수.
510
511     입력:
512     pdf_bytes:
513         FastAPI에서 await file.read()로 읽은 파일 bytes.
514         PDF뿐 아니라 pymupdf가 지원하는 모든 포맷(pptx, docx, hwp 등) 가능.
515
516     zoom:
517         PDF 렌더링 확대 비율.
518         OCR 품질을 위해 2 정도 추천.
519
520     filetype:
521         pymupdf.open()에 전달할 파일 타입 힌트.
522         기본값 "pdf". pptx/docx/hwp 등도 직접 지정 가능.
523
524     처리 흐름:
525     1. PDF 열기
526     2. 페이지 반복
527     3. native text 추출
528     4. image block 영역 추출
529     5. image block만 clip 렌더링
530     6. clip 이미지 OCR
531     7. native text + OCR text 병합
532     8. bbox 기준 정렬
533     9. 페이지별 document 생성
534
535     반환:
536     [
537         {
538             "page": 1,
539             "items": [...],

```

```

540         "text": "...
541     },
542     ...
543 ]
544 """
545
546 doc = pymupdf.open(
547     stream=pdf_bytes,
548     filetype=filetype
549 )
550
551 document = []
552
553 for page_idx, page in enumerate(doc):
554     page_number = page_idx + 1
555
556     print(f"\n===== PAGE {page_number} =====")
557
558     # =====
559     # 1. native text 추출
560     # =====
561     # PDF 내부에 텍스트 객체로 들어있는 글자를 추출한다.
562     # OCR보다 빠르고 정확하다.
563     native_blocks = extract_native_blocks(
564         page,
565         zoom=zoom
566     )
567
568     print("native:", len(native_blocks))
569
570     # =====
571     # 2. image block 영역 추출
572     # =====
573     # PDF 안에 포함된 이미지 영역만 가져온다.
574     # 이 영역만 OCR하면 전체 페이지 OCR보다 훨씬 빠르다.
575     image_regions = extract_image_regions(
576         page,
577         zoom=zoom
578     )
579
580     print("image regions:", len(image_regions))

```

구현 의의

- 사용자 문서 업로드부터 텍스트 추출, 요약, 법률 분류, 청킹, 임베딩까지 하나의 처리 흐름으로 연결
- native text 우선·이미지 영역 OCR 보완 방식으로 불필요한 전체 페이지 OCR 비용을 절감
- 파일 형식별 오류를 구분해 지원 불가 문서와 서버 오류를 사용자에게 명확하게 안내

10. 단위 테스트 및 구현 요약

대상 파일

```

Back-end/tests/unit/test_vector_service.py
Back-end/tests/unit/test_chat_service.py
Back-end/tests/unit/test_prompt_service.py
Back-end/tests/unit/test_hallucination_service.py
Back-end/tests/unit/test_service.py

```

검증 범위

- 검색 결과 정규화, 임베딩 객체/dict 호환, vector_id 기반 RDB 본문 결합
- general/document/hybrid 검색 모드와 사용자 2건·공용 3건 선택 규칙
- 후속 질문 검색어 생성, 비법률 점수 차단, LLM 호출 흐름

- 이전 대화·참고자료·현재 질문 프롬프트 조립
- 답변 조문 추출·검증, 경고 생성, 출처 응답 포맷
- Ollama 요청 payload, HTTP 오류 및 timeout 처리

테스트 결과 pytest 기준 총 46건 통과, 실패 0건. Pydantic 설정 방식과 관련된 경고 4건은 기능 실패가 아닌 향후 설정 마이그레이션 점검 항목으로 분리했습니다.

핵심 구현 수치

검색 후보: Top-K 20

최종 모델 입력: 5개 참고자료

Hybrid 비율: 사용자 문서 2 + 공용 법률 3

벡터 가중치: Dense 0.7 + Sparse 0.3

법률 최소 점수: 0.99

비법률 차단 점수: 1.3

모델 컨텍스트: 16,384 tokens

최대 생성: 300 tokens

구현 성과

- 벡터 검색 결과와 RDB의 실제 청크 본문을 연결해 모델이 메타데이터가 아닌 원문을 참고하도록 구성
- 파일 업로드 여부에 따라 일반 검색과 Hybrid 검색을 자동 전환해 개인 문서와 공용 법률 자료를 함께 활용
- 후속 질문은 직전 질문을 이용해 재검색하되, 이전 답변은 검색 근거에서 제외해 검색 오염을 완화
- 법령명·조문·수치 생성 제한과 사후 조문 검증을 결합해 환각 위험을 사용자에게 명시
- 외부 연동은 Mock으로 분리해 핵심 서비스 로직을 독립적으로 반복 검증할 수 있는 단위 테스트 기반 마련